

Shihan Sharar

☎ (548) 577-1483 | ✉ ssharar@uwaterloo.ca | 🏠 s-sharar.vercel.app | 🌐 s-sharar | in shihan-sharar

Education

University of Waterloo

Bachelor of Computer Science

Waterloo, ON

Sept 2023 - Aug 2027 (Expected)

- **Dean's List** (all terms); CGPA: **94%**, Faculty GPA: **96%** (with advanced courses)
- **Clubs:** Measuring rover inclination angle for UW Robotics Club, member of UW Quant and UW Putnam Club
- **Skills:** C++, C, C#, Java, Go, Python, React, Rust, Typescript/Javascript, Node.js, Next.js, CSS

Experience

University of Waterloo

Undergraduate Research Assistant - Prof. Y. Zhang

Waterloo, ON

Dec 2025 - Present

- Building a probabilistic programming language (PPL) compiler that bit-blasts continuous distributions into discrete Bernoulli random variables for efficient inference.

BitGo

Software Engineer Intern - Prime Trade

Palo Alto, California

Sep 2025 - Dec 2025

- Designed and engineered an **LP-settlement** tracking system in **Go**, introducing new data models in **PostgreSQL**, **GraphQL** schemas/resolvers, and APIs to track settlement lifecycle state and reconcile trades.
- Developed an ingestion framework for **Talos** OTC fills, producing normalized data for the settlement system.
- Built **React** dashboards with filtering and sorting over ingestion and settlement data, cutting operational settlement workflows by **~20 minutes** per case; validated with **Vitest**.
- Implemented a notification pipeline, defining **gRPC** schemas, producers that write to an **outbox**, a publisher that streams events to **Kafka**, and consumers that trigger **Novu** workflows for over **10,000** notifications daily.
- Eliminated duplicate notifications and hardened delivery across failures using deterministic **idempotency** keys, persisted delivery state, and exponential-backoff retries with a DLQ.

University of Waterloo

Undergraduate Research Assistant - Prof. N. Asokan

Waterloo, ON

May 2025 - Aug 2025

- Engineered a **C++** command-line interface using the **llama.cpp** API, adding an executor framework that maps model outputs to deterministic system actions with full control-flow tracing.
- Built a secure-world **OP-TEE** monitor in **C** (emulated ARM TrustZone) to validate all LLM interactions via signing/hashing, preventing indirect prompt-injection and enforcing control-flow integrity (CFI).

Ford Motors Canada

Platform Software Developer - Provisioning

Ottawa, ON

Jan 2025 - Apr 2025

- Boosted unit-test coverage for the provisioning component from **95.0%** to **99.6%** line coverage and from **92.5%** to **98.7%** decision coverage using **C++** and **GTest/GMock**, while driving code-duplication down from **0.5%** to **0%**.
- Engineered a helper class for async invocation of component-proxy methods, boosting service responsiveness.
- Integrated **Valgrind** & **sanitizer** tooling into the **Python**-based functional-test suite, automating detection of CPU bottlenecks & memory leaks, and analyzing the resulting logs to eliminate **20+** issues.
- Eliminated **120+** code smells and slashed duplication from **20%** to **0%** for the Android provisioning component.

Projects

Velocore: Multithreaded C++ Trading Engine with React Visualization



- Built a multithreaded trading engine using **Crow**, **Boost.Asio**, & **Boost.Beast** for async I/O, & websocket communication; parsing **1,000+** orders in under **30ms** with low-latency trade execution and health-check endpoints.
- Integrated **Alpaca's** market-data feed and paper-trading **REST API** into a custom broker interface for real-time quote ingestion, order management, and portfolio tracking.
- Engineered a **React/Next.js** frontend with real-time order entry and live order-book/portfolio visualizations.

RAllnet: C++ Multiplayer Strategy Game



- Engineered a **C++** strategy game by applying **MVC** architecture, **SOLID** principles (SRP), and the **Observer** design pattern to decouple core logic from both text-based & **X11** graphical interfaces.
- Scaled gameplay to 2- and 4-player modes with dynamic board layouts and automated elimination cleanup.